

AIRBNB DATASET

BY

Talha Murtaza

Artificial Intelligence Module

Lecturer: Dr. Forooz Shahbazi Avarvand

GIT LINK: <https://github.com/talhamch3/AI-Airbnb-project>

Table of Contents

Abstract:	2
Introduction:	3
Dataset:	3
Listings:	3
Calendar:	3
Reviews:	4
Data Preprocessing & Feature Engineering:	4
Data Cleaning:	4
Feature Engineering:	4
Final Feature Set:	4
Model Development & Experiment Tracking:	5
Model Experiments:	5
Experiment Tracking with MLflow:	5
Model Registry:	5
Model Deployment:	5
FASTAPI Web Service:	5
Batch Prediction Pipeline:	6
Reproducibility through Docker:	6
Dockerfile:	6
Docker Compose:	6
Monitoring Framework:	7
Batch Generation:	7
Fake Batch Generation:	7
Model Performance Monitoring:	7
Seasonal Monitoring:	7
Listing-Level Monitoring:	7
Database Architecture:	8
model_monitoring:	8
seasonal_availability:	8
listing_metrics:	8
Dashboard Visualization:	9
Continuous Integration & Continuous Deployment:	10
Automated Testing:	10
Automated Deployment:	10
Limitations & Future Work:	11
Conclusion:	11
Tool Usage & AI Disclosure:	11

Abstract:

The rapid growth of online short-term rental platforms has increased the need for accurate and scalable pricing systems. Airbnb hosts must continuously adjust property prices based on property characteristics, demand patterns, availability and customer engagement. Traditional machine learning models focus exclusively on model development but overlook deployment, reproducibility, monitoring, and automation requirements necessary for the production environment.

This project presents a complete MLOps framework for Airbnb price prediction using the Seattle Airbnb dataset. This project includes automated data preprocessing, experiment tracking, model training, model registry management, deployment through FastAPI, containerization using Docker, monitoring through PostgreSQL and Grafana, and automated testing and deployment through GitHub Actions CI/CD pipelines.

Introduction:

Airbnb hosts often face challenges when deciding appropriate prices for their properties. Pricing a listing too high can result in reduced bookings, while pricing it too low can result in lost revenue because pricing is influenced by multiple factors such as property size, availability, location, and customer demand, this project offers a practical solution for estimating optimal prices.

The primary objective of this project is not only to build an accurate Airbnb price prediction model but also to develop a complete MLOps framework that supports reproducibility, deployment, monitoring, and automation.

Dataset:

The project utilizes the Seattle Airbnb Open Dataset consisting of three primary files:

Listings:

This dataset contains detailed information about Airbnb properties including:

- Listing ID
- Number of bedrooms
- Number of bathrooms
- Property characteristics
- Price information

This file serves as the primary source of property information and contains the target variable used for model training.

Calendar:

The calendar dataset contains daily availability information for each listing. Each record indicates whether a property was available on a particular day. This dataset is used to calculate the Availability Rate for the project. Availability rate provides insight into property utilization and booking behavior.

Reviews:

The reviews dataset contains historical customer review information. This dataset is used to calculate the Review Count. Review count acts as a proxy indicator for customer demand and listing popularity.

Data Preprocessing & Feature Engineering:

Before training the model, the raw dataset will undergo our preprocessing pipeline and feature engineering steps.

Data Cleaning:

The listings dataset contains prices stored as text value containing currency symbols and commas. These values are converted into numerical formats to enable processing. Missing values are also identified and removed only when they affect critical features required for model training.

Feature Engineering:

The first feature engineered is availability rate where calendar dataset containing daily availability indicators are represented as t(available) and f(unavailable). These values are converted into numerical values 1(available) and 0(unavailable), and the average availability across all recorded days is calculated for each property.

The second feature is review count. The review dataset is grouped by listing ID. The number of reviews associated with each property is counted and the resulting feature is called review_count. This feature represents customer interaction and listing popularity.

Final Feature Set:

The final model is trained using the following features:

- bedrooms
- bathrooms
- availability_rate

- review_count

Our target feature is price, and these features are selected because they provide a combination of structural, operational, and customer-driven information.

Model Development & Experiment Tracking:

Model training is implemented within the training pipeline. The dataset is divided into a 80-20 split where 80% is our training data and 20% is our testing data. This separation allows the evaluation of model performance on unseen data.

Model Experiments:

We used Random Forest which builds multiple decision trees and combines their predictions. It is resistant to overfitting and robust. We observed a RMSE of 69.37 using this model.

We also used Gradient Boosting, which builds trees sequentially and continuously improves prediction errors from previous trees. It performs well on structured tabular data. We observed a RMSE of 68.94 using this model.

RMSE of both models indicate that their predictions are very close to each other but to shorten it down to one, we will choose the model with least RMSE which is gradient boosting.

Experiment Tracking with MLflow

We use MLflow throughout the project to maintain reproducibility because it tracks every run and records model parameters, evaluation metrics, trained model artifacts, and experiment metadata. All the metrics are stored within the experiment "Airbnb Price Prediction." Due to lower RMSE, Gradient Boosting will be selected as the production model.

Model Registry:

The selected model is registered within the MLflow Model Registry with the name of "AirbnbPriceModel." The registry provides model versioning, centralized storage, deployment management, and reproducibility. The monitoring and deployment pipeline always loads the latest registered version automatically.

Model Deployment:

Once we have our best performing mode, it is then deployed as a web service using FastAPI.

FASTAPI Web Service:

FastAPI provides a lightweight framework for exposing machine learning predictions through HTTP endpoints. When the service starts, MLFlow connection is established and the latest model is loaded from the registry. Once the model is loaded, API endpoints become available.

The primary endpoint is POST/predict where it accepts bedrooms, bathrooms, availability_rate and review_count. The model processes the request and returns a predicted Airbnb price.

Batch Prediction Pipeline:

In addition to real-time predictions, the project supports batch inference. The batch pipeline loads the latest registered model, generates prediction batches, produces prediction outputs, and saves the result to CSV files. This functionality is useful for large-scale analysis and scheduled prediction jobs.

Reproducibility through Docker:

One of the major objectives of MLOps is reproducibility. To achieve this, the entire project is containerized using Docker.

Dockerfile:

The Dockerfile defines the application environment.

It specifies:

- Python version
- Required libraries
- Application files
- Startup commands

By using Docker, every user runs the project within an identical environment regardless of operating system differences.

Docker Compose:

Docker compose orchestrates multiple services simultaneously. The project includes:

- MLflow Server is responsible for experiment tracking, model registry, and artifact management.
- Airbnb API is responsible for FastAPI deployment, prediction services, and monitoring execution.
- PostgreSQL is responsible for monitoring data storage and historical metric tracking.

- Adminer is responsible for database inspection, SQL query execution and table management
- Grafana is responsible for visualization, dashboards, and monitoring analytics.

Docker compose automatically creates networking between these services, allowing them to communicate without manual configuration.

Monitoring Framework:

Monitoring is implemented through a dedicated monitoring pipeline. The objective is to evaluate model performance continuously after deployment.

Batch Generation:

The monitoring pipeline creates a random sample of 100 listings; this batch simulates newly arriving production data. The monitoring cycle operates exclusively on this batch.

Fake Batch Generation:

A folder “Batch Generator” includes a fake csv file generator and a monitoring pipeline; both are used to generate a simulated batch and then store metrics to dataset for visualization in Grafana. The purpose of it is to strictly demonstrate data drift.

Model Performance Monitoring:

The deployed model generates predictions for the current batch, and the prediction errors are compared against actual prices. The resulting RMSE score is stored in PostgreSQL and enables long-term tracking of model performance. Increasing RMSE values may help identify data drift, model degradation, or market changes.

Seasonal Monitoring:

Monitoring pipeline also performs seasonal analysis by dividing the months into Winter, Spring, Summer and Autumn. Average availability is calculated for each season, and the results are stored in PostgreSQL and visualized in Grafana dashboards. This monitoring helps explain market behavior changes across different periods of the year.

Listing-Level Monitoring:

The monitoring pipeline also stores `listing_id`, `bedrooms`, and `availability_rate`. The database uses UPSERT operations to ensure that `listing_id` remains unique by preventing duplicate entries.

Database Architecture:

Monitoring metrics are stored within PostgreSQL, and three tables are maintained.

model_monitoring:

Stores:

- timestamp
- RMSE

Purpose:

Track model performance over time.

seasonal_availability:

Stores:

- season
- average availability
- timestamp

Purpose:

Track seasonal trends.

listing_metrics:

Stores:

- listing_id
- bedrooms
- availability_rate

Purpose:

Track listing-level changes.


Adminer 5.4.2

Schema: public

DB: airbnb
Schema: public

SQL command Import
Export Create table

select listing_metrics
select model_monitoring
select seasonal_availability

Alter schema Database schema Routines Sequences User types

Tables and views
Search data in tables (3)
LIKE %% Search

☐	Table	Engine	Data Length?	Index Length?	Data Free	Auto Increment	Rows?	Comment?
☐	listing_metrics	table	98,304	253,952	?	?	~ 1,038	
☐	model_monitoring	table	8,192	16,384	?	?	?	
☐	seasonal_availability	table	16,384	16,384	?	?	~ 12	
3 in total				en_US.utf8	122,880	286,720	0	

Dashboard Visualization:

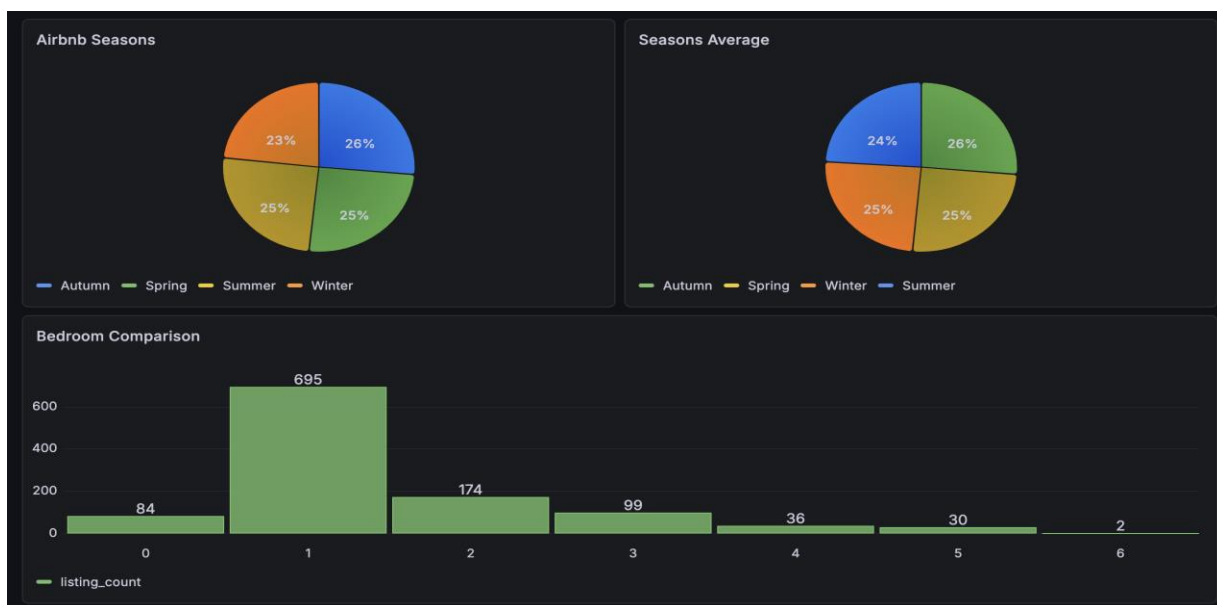
Grafana is connected directly to PostgreSQL.

The dashboard visualizes:

- RMSE trends
- Seasonal availability trends (Overall and batch both)
- Listing-level metrics

These dashboards provide operational visibility into the deployed model.

The monitoring system allows engineers to identify abnormal behavior before business performance is affected.





Continuous Integration & Continuous Deployment:

To improve reliability and reduce deployment risks, the project incorporates CI/CD automation through GitHub Actions.

Automated Testing:

We verify the critical functionalities using three tests

- Data loading tests ensure that all datasets exist, load successfully, and contain records.
- Seasonal Metrics test ensures that all four seasons exist, and seasonal calculations remain valid.
- Listing Metrics test ensures that all listing_id are unique and data is not duplicated.

Automated Deployment

When code is pushed to the repository:

- Tests are executed automatically.
- Build validation occurs.
- Docker images are created.
- Images are published to GitHub Container Registry.

This process reduces manual deployment effort and improves software quality.

Limitations & Future Work:

In this project, the basic limitation was the dataset; the provided dataset contains old trends and metrics but not current trends. If we had access to latest data, we could measure the data drift using Kolmogorov-Smirnov drift testing and do a comparison of both datasets to identify the shifting trends. The simulated batch data does not provide any credible information regarding trends or shifts in real time.

Future work must include a proper mechanism to compare models and choose the best one, as this does not exist in the project and although it works fine in this case, it should be automated. In CI/CD automation, due to the structure of monitoring pipeline, ci.yml had to run an empty mlflow container to avoid error, even in the experiments where mlflow was not required, further work could be done to enhance the structure of monitoring pipeline to avoid any conflicts.

Conclusion:

This project demonstrates the successful implementation of a complete MLOps lifecycle for Airbnb price prediction. Starting from raw Airbnb datasets, the framework performs preprocessing, feature engineering, model training, experiment tracking, deployment, monitoring, and automated delivery within a unified architecture.

The use of MLflow provides reproducible experiment management and model versioning. FastAPI enables real-time predictions, while Docker ensures deployment consistency across environments. PostgreSQL, Grafana, and Adminer provide operational visibility and monitoring capabilities. Finally, GitHub Actions automates testing and deployment processes through CI/CD workflows.

The resulting system moves beyond traditional machine learning experimentation and demonstrates how predictive models can be managed, monitored, and maintained in realistic production environments.

Tool Usage & AI Disclosure:

This project utilized open-source technologies and AI-assisted tools to support development, debugging, documentation, and technical discussions. All major design decisions, implementation, model development, deployment, monitoring, and evaluation were performed and validated by the project author. AI-generated suggestions were reviewed and verified before inclusion in the final system.